# Rapport de développement

# Jeu de combat au sein de l’engine Roblox

Travail remis à

Mr. Nicolas Bourré

420-1SH-SW

Par

Louis-Philippe L’Écuyer

Veille Technologique

Cégep de Shawinigan

12 juin 2025

Ce projet a été réalisé sur une période de deux semaines dans le cadre du cours de veille technologique. L'objectif principal était de concevoir un micro-projet basé sur une technologie avec laquelle je suis peu expérimenté, et qui n'avait pas été abordée au cours de mon parcours collégial. Parmi les options proposées, j'ai choisi de m'initier au développement d'un jeu 3D multijoueur en utilisant les services et les assets que le moteur de Roblox met à disposition. Cette décision m'a permis de découvrir plusieurs concepts avec lesquels je n'avais jamais pensé expérimenter, notamment la logique de synchronisation serveur-client, la gestion des animations, et l'architecture de projet via Rojo. Et l'utilisation des events pour relier les scripts clients et serveur.

Le projet visait à offrir une expérience interactive dans laquelle deux joueurs s'affrontent en temps réel dans une arène médiévale après avoir été appariés dans un lobby multijoueur par matchmaking. Chaque joueur choisit une classe de personnage, est ensuite téléporté dans une arène, puis engage un duel skill-based jusqu'à ce que l'un d'eux perde toutes ses vies.

Ce projet m'a amené à comprendre et intégrer plusieurs composantes essentielles du développement sur Roblox, notamment :

- La différence entre les scripts locaux, serveur et module
- La communication entre client et serveur via des RemoteEvents
- Les modules pour la communication des scripts de même type
- Travailler dans un environnement 3D
- Utiliser les services intégrés par Roblox
- Utiliser un langage de programmation non orientée objet pour un jeu
- L'animation des personnages via des modèles 3D
- La gestion des états d'un joueur (vie, mort, victoire)
- L'intégration d'animations synchronisées visibles par tous les joueurs
- L'utilisation du plugin Rojo pour pouvoir synchroniser mon code avec GitHub

Le jeu comprend plusieurs fonctionnalités essentielles qui permettent une expérience fluide et engageante. D'abord, un système de matchmaking automatisé regroupe les joueurs par paires à l'aide d'une file d'attente gérée côté serveur. Une fois deux joueurs appariés, ils sont téléportés ensemble dans une arène de combat grâce au TeleportService, avec transfert des données concernant leur classe respective.

Une fois dans l'arène, une animation joue dépictant les joueurs et leurs classes de manière cinématographique. Ensuite, les personnages peuvent se déplacer librement dans l'espace

3D. Le système de combat repose sur des animations d'attaques enchaînables, divisées en trois étapes distinctes. Le joueur peut effectuer un combo fluide en activant ses attaques au bon moment, à l'intérieur de fenêtres temporelles précises détectées grâce aux AnimationKeyframeEvents. Cette approche favorise le timing plutôt que le spam de touches, et donne une sensation de contrôle plus stratégique. Les joueurs ont également accès à un système de dash directionnel dans quatre directions (avant, arrière, gauche, droite), rechargeables aux 2 secondes pour éviter l'abus et garder le jeu compétitif, asinsi qu'un systeme de parry, qui inflige un movement lock sur le personnage adverse s'il frappe le joueur qui parry. Le parry, cependant, lock le joueur l'utilisant sur une courte période. C'est donc un outil à timing. Chaque joueur commence la partie avec trois vies. Lorsqu'un joueur est vaincu, les deux joueurs sont téléportés à leurs spawns respectifs. Si ses vies tombent à zéro, une victoire est automatiquement déclarée, et une interface utilisateur affiche le nom du joueur gagnant. Finalement, une interface dynamique affiche en tout temps le nombre de vies restantes. Cette interface est injectée manuellement dans le PlayerGui.

Ce projet m'a permis de sortir de ma zone de confort en explorant un environnement de développement totalement nouveau. En seulement deux semaines, j'ai pu assimiler plusieurs notions techniques essentielles liées à la programmation de jeux multijoueurs et à la communication client-serveur dans Roblox. J'ai découvert des outils comme Rojo qui a grandement facilité la récupration des données. Concevoir un jeu complet, aussi simple soit-il, m'a donné une perspective concrète sur le fonctionnement réel d'un moteur de jeu multijoueur et les contraintes qui viennent avec le développement temps réel et multijoueur.

En conclusion, ce projet a été non seulement enrichissant sur le plan académique, mais aussi très motivant sur le plan personnel. Il m'a donné envie de poursuivre mes apprentissages dans le domaine du développement de jeux vidéo.

Références :

- Accueil page de documentation : https://create.roblox.com/docs/creation

- Types de scripts (server, local, module) : https://create.roblox.com/docs/scripting/locations

- Camera location : https://create.roblox.com/docs/en-us/workspace/camera

- UI : https://create.roblox.com/docs/en-us/ui

- Cframes (movements et rotations) : https://create.roblox.com/docs/en-us/workspace/cframes#cframe-basics

- Collision event : https://create.roblox.com/docs/en-us/workspace/collisions

- Bindable events pour lier le client et le serveur : https://create.roblox.com/docs/scripting/events/bindable

- Physics overview : https://create.roblox.com/docs/en-us/physics

- Player overview : https://create.roblox.com/docs/players

- Avatar editor pour modifier l'apparence du personnage in-game : https://create.roblox.com/docs/characters/appearance

- Animation editor : https://create.roblox.com/docs/animation/editor

- Teleport service pour changer de "scene" : https://create.roblox.com/docs/projects/teleport